Artefact — AI Engineer Technical Case

Candidate: Anderson Martins | **Role:** AI Engineer – Full-Stack para Aplicações com IA Generativa
GitHub Repository: https://github.com/snagh/emporio-da-musica-ai-agent
Date: October 2026 | **Evaluation Language:** English

---

1. Executive Summary & Problem Formulation

This project delivers an end-to-end, production-ready AI Customer Support Agent for **Empório da Música**, a specialized musical instrument store located in Campo Grande/MS. The store's human staff was heavily overloaded with repetitive inquiries: catalog checks, instrument specifications, stock availability, order tracking, store policies, and operating hours. The solution implements an agentic architecture combining **Deterministic Tool Calling** over in-memory relational data with **Semantic RAG (Retrieval-Augmented Generation)** over the store's policy guidelines.

2. Architectural Design & Technical Decisions

• **Decoupled Hybrid Engine:** Rather than exposing raw Text-to-SQL (which risks SQL injection and schema hallucination), we designed strongly typed Pydantic tools for structured queries (search_catalog, get_order_status, list_active_promotions) and a semantic retriever for unstructured policies (search_policies).
• **In-Memory SQLite (:memory:):** Operational data spans 6 CSV tables (categories, products, customers, orders, items, promos). Ingesting them into an in-memory SQLite store provides sub-millisecond (< 1 ms) query times without requiring external databases.
• **Zero-Dependency BM25 Okapi RAG:** The provided store policy PDF presented whitespace corruption upon extraction. We built an automated text stream normalizer and a high-performance BM25 Okapi retriever in pure Python. It requires zero API tokens, ensuring instant, cost-free local evaluation.
• **Multi-Model Agnostic Support:** Native integration with Google Gemini (gemini-2.5-flash via google-genai) and OpenAI (gpt-4o-mini), paired with an autonomous local fallback mode for offline testing.
• **Interactive Terminal CLI:** Developed using rich, featuring formatted panels, live tool-execution feedback, and conversation export.

3. Exploratory Data Analysis & Business Insights

Through scripts/inspect_data.py, we audited 100% of the relational data integrity:
• **Relational Keys:** 0 orphan records across all tables (products ↔ categories, order_items ↔ orders ↔ customers).
• **Catalog & Pricing:** 65 products registered (62 active, 3 inactive) spanning from R$ 159.90 to R$ 19,567.00. Crucially, the specs column contains JSON-serialized tonewood and hardware attributes (e.g. solid Spruce tops, nylon strings).
• **Active Promotions:** Out of 25 historical campaigns, exactly 4 are currently active (#90, #94, #121, #127).
• **Payment Terms:** 5% instant discount on PIX; interest-free credit card installments up to 12x (minimum R$ 100/installment).

4. Domain Guardrails & Store Persona

• **Golden Commercial Rule:** The store exclusively sells complete instruments and **does NOT sell standalone accessories** (individual string packs, loose guitar picks, cables, strap-locks, standalone cases, or amplifiers). The agent strictly enforces this by cordially declining accessory requests and proposing related instruments.
• **7-Day Legal Return Window:** Applies the Brazilian Consumer Defense Code (CDC) right of withdrawal for online orders (7 calendar days).
• **Out-of-Scope Guardrails:** Politely rejects off-topic queries (political commentary, culinary recipes, generic code generation) while steering the conversation back to musical instruments.

5. Automated Test Suite (100% Passing)

A comprehensive test suite of **31 automated unit tests** was developed with pytest, executing in 2.2 seconds:

Test Module	Focus Area	Test Count	Status
test_database.py	SQLite Ingestion, Joins & Pricing Calculations	9	PASSED (100%)
test_tools.py	Pydantic Schemas & Execution Dispatcher	9	PASSED (100%)
test_rag.py	PDF Normalization, Chunking & BM25 Retrieval	6	PASSED (100%)
test_agent.py	Multi-turn Memory, Tool Routing & Guardrails	7	PASSED (100%)

6. Validated Interaction Scenarios (Edge Cases)

The repository includes 4 markdown transcripts in examples/ demonstrating real edge-case handling:
• **Scenario 1 (Catalog & Price Filtering):** Customer requests beginner guitars under R$ 1,000; agent returns Yamaha C40 and Tagima Memphis with nylon string specs.
• **Scenario 2 (Policies & Returns):** Customer asks for store location in Campo Grande (Rua 14 de Maio, 3200) and details the 7-day online return policy.
• **Scenario 3 (Order Tracking):** Real-time lookup of Order #1, retrieving carrier tracking code (BRAB1234567BR) and delivery status.
• **Scenario 4 (Complex Multi-Constraint Case):** Customer asks for loose Fender picks (agent politely declines per accessory policy), then asks for active guitar promos, and calculates final checkout price with 5% PIX discount.

7. AI-Assisted Workflow Transparency (Artefact Compliance)

In full alignment with Artefact's instructions encouraging transparent use of AI tools:
• **Tool Employed: Google Antigravity** (agentic pair programming assistant powered by Gemini).
• **How AI was leveraged:** Expediting initial boilerplate code, discovering whitespace anomalies in the PDF extraction stream, and writing automated test assertions for edge cases.
• **Human Engineering Decisions:** Choosing an in-memory hybrid architecture over fragile SQL generation, structuring the BM25 Okapi semantic search algorithm, enforcing the store's accessory boundary, and authoring atomic Git commits across the development lifecycle.